

Week 4 Study Guide

Object-Oriented Programming in Python

This week you move from writing scripts to building **applications**. You will learn to design your own types of objects, combine them, and organise them into a small project. These are the same ideas behind FastAPI models, database tables, and agents and tools, which are coming in the next weeks.

	Classes and objects	Composition and inheritance
Topics	Classes and objects, attributes and methods, <code>__init__</code> , instance vs class attributes, small classes working together	Composition, inheritance basics, method overriding, reusable class design, project structure
You build	A shopping cart: <code>Product</code> , <code>CartItem</code> , <code>Cart</code>	A multi-file online store: products, cart, customer, orders

Each part follows the same rhythm: **Concept, Example, Your Turn, Challenge, Checkpoint**. Checkpoint answers are at the end. Practice files go in a `practice/` folder inside your uv project; the final project lives in `src/online_store/`.

Classes and objects

By the end of this class you can:

- Explain why object-oriented programming helps in larger applications.
- Write a class, create objects from it, and explain the difference between the two.
- Use attributes, methods, `__init__` and `self` correctly.
- Tell instance attributes from class attributes, and avoid the shared-list trap.
- Connect small classes together and see how this matches API and agent components.

Part 1 | Why classes?

CONCEPT

So far we stored related data in separate variables or dictionaries and wrote separate functions to work on them. That is fine for a small script. In a real application it breaks down:

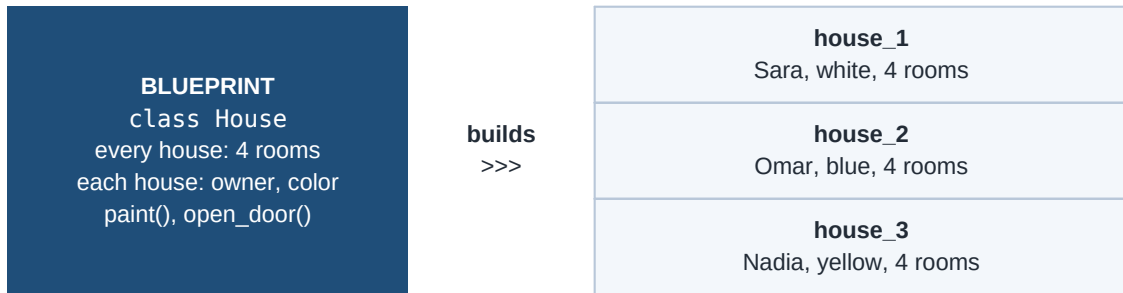
- The **data and the functions that use it are not connected**. Nothing says `sell()` belongs to a product.
- **Typos fail late**. `product["stok"]` is a `KeyError` at runtime, in the middle of your program.
- **Nothing guarantees the shape**. One dictionary may have `price`, the next may forget it.

A **class** bundles data (attributes) and behaviour (methods) into one unit with a guaranteed shape.

The blueprint idea

Picture an architect's **blueprint** for a house. The blueprint is not a house. You cannot live in it. It fixes the **structure**: every house built from it has the same layout, the same four rooms, the same doors and windows. From **one blueprint you can build many houses**, and what differs is the finishing: one is painted white, one blue, and each has its own owner. Each house is a separate, real thing.

Programming works the same way. A **class** is the blueprint. An **object** is one house built from it.



One blueprint, three different houses. Each house has its own values, but all follow the same design.

Word	Think of it as	In code
class	The architect's blueprint	<code>House</code>
object / instance	One real house built from the blueprint	<code>house_1</code>
attribute	A detail that can differ from house to house: its owner, its colour	<code>house_1.color</code>
class attribute	A fact fixed by the blueprint, the same for every house: the number of rooms	<code>House.rooms</code>
method	Something a house can do: open the door, get painted	<code>house_1.paint("green")</code>
constructor	The building work that creates a new house from the blueprint	<code>House("Sara", "white")</code>
self	"This particular house", the one the action is happening to	<code>self.color</code>

Our store project uses exactly the same idea: **Product** is the blueprint, and the Notebook, the Backpack and the Headphones are three different products built from it.

Why use OOP?

- **Modularity.** Each class has one job (**Product**, **Cart**, **Customer**), so a big program becomes small pieces that you can write and test one at a time.
- **Reusability.** Write the **Product** blueprint once and create thousands of products from it. The same class can move into another project.
- **Maintainability.** If the discount rule changes, you fix it in one class instead of hunting through the whole script.
- **Scalability.** A new kind of product (say clothing) does not force you to rewrite the cart.
- **Real-world modelling.** Products, customers and orders match the words people use about the business, so code and conversation line up.

This is why FastAPI, SQLAlchemy and agent libraries are all built from classes.

Four ideas behind OOP: a map for the week

Idea	In plain words	House analogy	You will meet it
Encapsulation	Keep data and the methods that use it together, and control how outside code touches it	You enter through the front door, not through the walls	Parts 2 and 9
Abstraction	Offer a simple way to use something and hide the details	You flip a switch without knowing the wiring	Parts 4 and 9
Inheritance	A new class reuses and extends an existing one	A villa blueprint starts from the standard house blueprint	Part 7
Polymorphism	The same method name behaves differently depending on the object	"Open the door": a key for one house, a remote for another	Part 7

EXAMPLE

One blueprint, many houses

practice/house.py

```
class House:
    rooms = 4 # fixed by the blueprint (you will meet this in Part 3)

    def __init__(self, owner, color):
        self.owner = owner # differs from house to house
        self.color = color

    def paint(self, new_color):
        self.color = new_color

    def open_door(self):
        print(f"{self.owner} opens the front door.")

    def __str__(self):
        return f"{self.owner}'s {self.color} house with {self.rooms} rooms"

def main():
    # one blueprint (the class), three houses (objects): same structure, own owner and colour
    house_1 = House("Sara", "white")
    house_2 = House("Omar", "blue")
    house_3 = House("Nadia", "yellow")

    house_2.paint("green") # repaints house_2 only
    house_3.open_door()
    print(house_1)
    print(house_2)
    print(house_3)

if __name__ == "__main__":
    main()
```

OUTPUT

```
Nadia opens the front door.
Sara's white house with 4 rooms
Omar's green house with 4 rooms
Nadia's yellow house with 4 rooms
```

Compare `house_1` and `house_2` in the output: painting `house_2` did not change `house_1`. They were built from the same blueprint, but they are **separate objects**.

The same idea in a shop: the dictionary way (what we are moving away from)

practice/dict_version.py

```
notebook = {"name": "Notebook", "price": 4.50, "stock": 50}
backpack = {"name": "Backpack", "price": 35.00, "stock": 12}

def sell(product, quantity):
    if quantity > product["stock"]:
        print("Not enough stock")
    else:
        product["stock"] -= quantity

sell(notebook, 10)
print(notebook["stock"])
```

OUTPUT

40

YOUR TURN

1. Type `practice/house.py` and run it. Then build a **third** house, repaint it, and confirm that the other two houses keep their colours.
2. On paper or in a notes file, fill in the table: for each real-world thing, write **three attributes** (data it holds) and **two methods** (things it can do). Keep your answers: you will turn one of them into code in Part 2.

Thing	3 attributes (data)	2 methods (actions)
Book		
BankAccount		
Song		

CHECKPOINT

1. Give two problems with storing each product as a dictionary.
2. Explain the difference between a class and an object using the blueprint and the house.
3. After `house_2.paint("green")`, why does `house_1.color` stay `"white"`?
4. Name the four ideas behind OOP and match one of them to a house example.

Part 2 | Your first class

CONCEPT

Here is the anatomy of a class. Read it top to bottom:

Anatomy

```
class Product:                                # 1. class name: CapitalizedWords
    def __init__(self, name, price, stock):    # 2. runs automatically on Product(...)
        self.name = name                     # 3. attribute = value, stored ON the object
        self.price = price
        self.stock = stock

    def is_available(self):                    # 4. a method: first parameter is always self
        return self.stock > 0
```

- `Product("Notebook", 4.50, 50)` creates a new object and runs `__init__` for you.
- `self` is the object being created or used. **You never pass it yourself.** Python does. `notebook.sell(10)` is really `Product.sell(notebook, 10)`.
- In `self.name = name`, the left side is the attribute that lives on the object; the right side is the parameter that was passed in.
- `__str__` is optional. It controls what `print(obj)` shows.

EXAMPLE

A complete Product

practice/product.py

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def is_available(self):
        return self.stock > 0

    def sell(self, quantity):
        if quantity > self.stock:
            print(f"Only {self.stock} of {self.name} left.")
            return False
        self.stock -= quantity
        return True

    def __str__(self):
        return f"{self.name} - ${self.price:.2f} ({self.stock} in stock)"

def main():
    notebook = Product("Notebook", 4.50, 50)
    backpack = Product("Backpack", 35.00, 12)

    print(notebook)
    notebook.sell(10)
    print(notebook)
    print(notebook.sell(100))
    print(backpack.is_available())

if __name__ == "__main__":
    main()
```

OUTPUT

```
Notebook - $4.50 (50 in stock)
Notebook - $4.50 (40 in stock)
Only 40 of Notebook left.
False
True
```

Without `__str__`, printing an object shows something unhelpful:

practice/no_str.py

```
class Product:
    def __init__(self, name):
        self.name = name

print(Product("Notebook"))
```

OUTPUT

```
<__main__.Product object at 0x7f3a2c1b5d90>
```

You have been using objects all along. Strings and lists are objects too, and the dot calls their methods:

practice/everything_object.py

```
name = "notebook"
prices = [4.50, 35.00]

print(type(name))
print(name.upper())           # a method called on a string object
prices.append(59.99)          # a method called on a list object
print(prices)
print(type(5))
```

OUTPUT

```
<class 'str'>
NOTEBOOK
[4.5, 35.0, 59.99]
<class 'int'>
```

In Python, numbers, strings, lists and even functions are objects. Classes just let you create your own kinds. That idea returns when functions become tools for an agent.

WATCH OUT Inside `__init__`, write `self.price = price`, not just `price = price`. The second form only assigns a local variable that disappears when the method ends. The object never gets a `price` attribute.

YOUR TURN

1. Create `practice/book.py` with a `Book` class (title, author, pages) and a `__str__` method. Create two books and print them. Use your paper table from Part 1 for ideas.
2. Add two methods to `Product`: `restock(quantity)` adds to the stock, and `apply_discount(percent)` lowers the price. Test with a Notebook (4.50, stock 50): after `restock(10)` and `apply_discount(10)`, printing it should show `Notebook - $4.05 (60 in stock)`.

CHALLENGE

Add `total_value()` (price multiplied by stock) and `is_low_stock(limit=5)` (True when the stock is at or below the limit).

CHECKPOINT

1. What does `self` refer to inside a method?
2. When exactly does `__init__` run?
3. What happens if you define `def is_available():` without `self`? (Try it and read the error.)

Part 3 | Instance attributes vs class attributes

CONCEPT

- **Instance attributes** (`self.x = ...` inside `__init__`) belong to **one object**. Every object has its own copy.
- **Class attributes** (written directly in the class body) belong to the **class** and are **shared** by every object. Use them for settings, constants and counters.
- Reading `obj.attr` looks on the object first, then on the class.
- Assigning `obj.attr = value` **always** creates or changes an instance attribute. To change the shared one, assign through the class: `Product.attr = value`.

NOTE

House analogy: the **number of rooms** is fixed by the blueprint, so every house has the same four. That is a class attribute. The **colour** and the **owner** belong to one house only. Those are instance attributes.

EXAMPLE

practice/class_attrs.py

```
class Product:
    currency = "USD"          # class attribute: one copy, shared by all
    product_count = 0

    def __init__(self, name, price, stock):
        self.name = name      # instance attributes: each object has its own
        self.price = price
        self.stock = stock
        Product.product_count += 1

    def __str__(self):
        return f"{self.name} - {self.price:.2f} {self.currency}"

notebook = Product("Notebook", 4.50, 50)
backpack = Product("Backpack", 35.00, 12)

print(Product.product_count)
print(notebook)
Product.currency = "PKR"     # change the class attribute
print(notebook)
print(backpack)

notebook.currency = "EUR"    # creates an instance attribute on notebook only
print(notebook)
print(backpack)
```

OUTPUT

```
2
Notebook - 4.50 USD
Notebook - 4.50 PKR
Backpack - 35.00 PKR
Notebook - 4.50 EUR
Backpack - 35.00 PKR
```

Read the output carefully: the change to `Product.currency` reached both products, but `notebook.currency = "EUR"` created a private copy on `notebook` only.

WATCH OUT

The shared-list trap. Never put a list or dictionary at class level if each object should have its own. Put it in `__init__`.

practice/shared_list_bug.py (the bug)

```
class Shelf:
    products = [] # class attribute: shared by ALL shelves

    def add(self, product):
        self.products.append(product)

shelf_a = Shelf()
shelf_b = Shelf()
shelf_a.add("Notebook")
print(shelf_b.products)
```

OUTPUT

```
['Notebook']
```

practice/shared_list_fix.py (the fix)

```
class Shelf:
    def __init__(self):
        self.products = [] # instance attribute: each shelf has its own

    def add(self, product):
        self.products.append(product)

shelf_a = Shelf()
shelf_b = Shelf()
shelf_a.add("Notebook")
print(shelf_a.products)
print(shelf_b.products)
```

OUTPUT

```
['Notebook']
[]
```

YOUR TURN

1. Add a class attribute `tax_rate = 0.10` and a method `price_with_tax()`. Create two products, then run `Product.tax_rate = 0.15`. Both products should now use the new rate.
2. Replace the `limit` parameter of `is_low_stock` with a class attribute `low_stock_limit = 5`.

CHALLENGE

Write a `Shelf` class that holds products in a list. It needs `add(product)` and `total_value()`. Create two shelves and prove they do **not** share their products.

CHECKPOINT

1. After `Product.currency = "PKR"` and `notebook.currency = "EUR"`, what does `backpack.currency` print?
2. Why is `products = []` at class level risky?

Part 4 | Objects working together

CONCEPT

Objects can **hold other objects**. A **Cart** holds **CartItem** objects, and each **CartItem** holds a **Product**. Each class has one small job, and they cooperate by calling each other's methods. This is called **composition**, and it is how real applications are built.

Who holds what

```
Cart
  items = [ CartItem(quantity=2) --> Product("Notebook"),
            CartItem(quantity=1) --> Product("Backpack") ]

cart.total() asks each CartItem for its subtotal()
              each CartItem asks its Product for the price
```

EXAMPLE

practice/cart_demo.py

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

class CartItem:
    def __init__(self, product, quantity):
        self.product = product
        self.quantity = quantity

    def subtotal(self):
        return self.product.price * self.quantity

class Cart:
    def __init__(self):
        self.items = []

    def add(self, product, quantity=1):
        if quantity > product.stock:
            print(f"Cannot add {quantity} x {product.name}: only {product.stock} in stock.")
            return
        self.items.append(CartItem(product, quantity))

    def total(self):
        total = 0
        for item in self.items:
            total += item.subtotal()
        return total

notebook = Product("Notebook", 4.50, 50)
backpack = Product("Backpack", 35.00, 12)

cart = Cart()
cart.add(notebook, 2)
cart.add(backpack, 1)
cart.add(notebook, 100)
print(len(cart.items))
print(cart.total())
```

OUTPUT

```
Cannot add 100 x Notebook: only 50 in stock.
2
44.0
```

Notice that `Cart` never calculates a price itself. It asks the `CartItem`, which asks the `Product`. Each object is responsible for what it knows best.

YOUR TURN

Add `count_items()` to `Cart`. It returns the total number of units in the cart (not the number of lines). With 2 notebooks and 3 backpacks in the cart it should return 5.

CHALLENGE

Add `remove(product)` to `Cart`. It removes that product's line and returns `True`, or returns `False` if the product is not in the cart.

CHECKPOINT

1. Which class calculates a subtotal, and why is that the right place for it?
2. In the demo, why does `len(cart.items)` print 2 and not 3?

Part 5 | Practice project: shopping cart

PROJECT

Finish the cart. Save the starter below as `practice/shop.py`. `Product` and `main()` are given; you complete the six `TODOs`. Your output must match the expected output exactly.

practice/shop.py (starter)

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def sell(self, quantity):
        if quantity > self.stock:
            print(f"Only {self.stock} of {self.name} left.")
            return False
        self.stock -= quantity
        return True

    def __str__(self):
        return f"{self.name} - ${self.price:.2f} ({self.stock} in stock)"

class CartItem:
    def __init__(self, product, quantity):
        # TODO 1: store the product and the quantity
        ...

    def subtotal(self):
        # TODO 2: return the product's price multiplied by the quantity
        ...

class Cart:
    def __init__(self):
        self.items = []

    def add(self, product, quantity=1):
        # TODO 3: refuse (print a message, return False) if the quantity is more
        #         than the stock; if the product is already in the cart, increase
        #         that item's quantity; otherwise append a new CartItem.
        #         Return True when something was added.
        ...

    def total(self):
        # TODO 4: add up the subtotal of every item and return the result
        ...
```

```

def show(self):
    # TODO 5: print every item and the total (match the expected output)
    ...

def checkout(self):
    # TODO 6: call sell() for every item, empty the cart, return the total
    ...

def main():
    notebook = Product("Notebook", 4.50, 50)
    backpack = Product("Backpack", 35.00, 12)
    headphones = Product("Headphones", 59.99, 4)

    cart = Cart()
    cart.add(notebook, 3)
    cart.add(backpack, 1)
    cart.add(headphones, 10)    # too many - should be refused
    cart.add(notebook, 2)      # same product again - quantity grows
    cart.show()

    paid = cart.checkout()
    print(f"Paid: {paid:.2f}")
    print(notebook)
    print(backpack)

if __name__ == "__main__":
    main()

```

Rules for TODO 3: refuse when the quantity is more than the stock, **and** when adding to an item already in the cart would take the combined quantity over the stock. Compare products with **is** (same object), not by name.

Expected output

uv run practice/shop.py

```

Cannot add 10 x Headphones: only 4 in stock.
--- Cart ---
5 x Notebook          22.50
1 x Backpack          35.00
Total                 57.50
Paid: 57.50
Notebook - $4.50 (45 in stock)
Backpack - $35.00 (11 in stock)

```

TIP

Get one TODO working at a time and run the file after each. Use `print()` liberally while debugging, and read the traceback from the bottom up.

Reference card: classes and objects

You want to...	Write
Define a class	<code>class Product:</code>
Set up a new object	<code>def __init__(self, name, price): ...</code>
Store data on the object	<code>self.name = name</code>
Create an object	<code>notebook = Product("Notebook", 4.50, 50)</code>
Call a method	<code>notebook.sell(10)</code>
Control print() output	<code>def __str__(self): return ...</code>
Class attribute (shared)	<code>currency = "USD" (in the class body)</code>
Change a class attribute	<code>Product.currency = "PKR"</code>
Object holds other objects	<code>self.items = [] then self.items.append(CartItem(...))</code>

Errors you will meet (and what they mean)

Error message	Usual cause	Fix
<code>TypeError: Product.__init__() missing 1 required positional argument: 'stock'</code>	Created an object with too few arguments	Pass every argument <code>__init__</code> expects
<code>AttributeError: 'Product' object has no attribute 'nam'. Did you mean: 'name'?</code>	Typo in an attribute name	Fix the spelling
<code>AttributeError: 'Product' object has no attribute 'price'</code>	Wrote <code>price = price</code> in <code>__init__</code> (no <code>self</code> .)	Write <code>self.price = price</code>
<code>TypeError: Product.is_available() takes 0 positional arguments but 1 was given</code>	Method defined without <code>self</code>	Add <code>self</code> as the first parameter
<code>NameError: name 'stock' is not defined. Did you mean: 'self.stock'?</code>	Used an attribute without <code>self</code> . inside a method	Write <code>self.stock</code>
<code>TypeError: Product.shout() missing 1 required positional argument: 'self'</code>	Called a method on the class instead of an object	Create an object first, then call the method on it

Composition and inheritance

By the end of this class you can:

- Combine small classes into larger systems using composition.
- Use inheritance and method overriding, and know when inheritance is the wrong tool.
- Design classes that are small, validated and reusable.
- Organise a multi-class application into modules inside a uv package.

Part 6 | Composition in depth: building bigger objects

CONCEPT

Composition means building a bigger object out of smaller ones. The test is the phrase **"has a"**: a Customer **has a** Cart; a Cart **has** CartItems; an Order **has** a Customer and a list of items. Each class stays small, and you can change one without breaking the others. In the house analogy: a house **has** rooms, and a kitchen **has** a stove. You build the big thing out of smaller parts.

Today `Cart.checkout()` must also hand back what was bought, so an `Order` can record it. Change it to return two values:

`Cart.checkout()` (updated: returns the purchased items and the total)

```
def checkout(self):
    if not self.items:
        raise ValueError("Cart is empty")
    total = self.total()
    purchased = self.items
    for item in purchased:
        item.product.sell(item.quantity)
    self.items = []
    return purchased, total
```

EXAMPLE

`src/online_store/orders.py`

```
from online_store.cart import Cart

class Order:
    order_count = 0

    def __init__(self, customer, items, total):
        Order.order_count += 1
        self.order_id = Order.order_count
        self.customer = customer
        self.items = items
        self.total = total

    def print_receipt(self):
        print(f"=== Receipt #{self.order_id} ===")
        print(f"Customer: {self.customer.name} <{self.customer.email}>")
        for item in self.items:
            print(f"  {item.quantity} x {item.product.name:<12} {item.subtotal():>8.2f}")
        print(f"  {'Total':<16} {self.total:>8.2f}")

class Customer:
    def __init__(self, name, email):
        self.name = name
        self.email = email
        self.cart = Cart()          # a Customer HAS a Cart
```

```
def place_order(self):
    items, total = self.cart.checkout()
    return Order(self, items, total)
```

- `Customer.__init__` creates its own `Cart()`. Each customer needs a cart of their own; sharing one would mix up their items (the same trap as the shared list in Part 3).
- `place_order()` does not calculate anything itself. It asks the cart to check out and wraps the result in an `Order`.
- `Order.order_count` is a class attribute, so every order gets the next number.

YOUR TURN

Give `Customer` an `orders` list. Every time `place_order()` runs, store the new order in it. Then add `total_spent()`, which adds up the totals of all the customer's orders.

CHALLENGE

Add `show_history()` to `Customer`. It prints one line per order, for example `Order #1 - 9.00`.

CHECKPOINT

1. Name the "has a" relationship in each pair: Customer/Cart, Order/Customer, Cart/CartItem.
2. Why does `Customer.__init__` create `Cart()` itself instead of receiving one from outside?

Part 7 | Inheritance and method overriding

CONCEPT

Inheritance means "is a": a `PerishableProduct` is a `Product`. The subclass gets everything the parent has, and can add or change things.

- `class PerishableProduct(Product):` makes `Product` the parent.
- `super().__init__(name, price, stock)` runs the parent's constructor, so you do not repeat its code.
- **Overriding**: if the subclass defines a method with the same name, Python uses the subclass version.
- **Why it is powerful**: code that calls `product.final_price()` does not care which kind of product it has. `CartItem.subtotal()` now uses `final_price()`, and the whole cart works with every new product type without any other change. One method name, many behaviours: this is **polymorphism**.

The hierarchy

```

      Product
     /      \
PerishableProduct  ElectronicsProduct
  days_left      warranty_months
  overrides final_price() and __str__()  overrides __str__()
  
```

NOTE

House analogy: an architect can start from the standard house blueprint and create a **villa** blueprint: "everything a house has, plus a swimming pool". A villa is a house, so it reuses the original design and adds or changes only what is different. That is inheritance.

EXAMPLE

src/online_store/models.py

```
class Product:
    currency = "USD"

    def __init__(self, name, price, stock):
        if price < 0:
            raise ValueError("Price cannot be negative")
        if stock < 0:
            raise ValueError("Stock cannot be negative")
        self.name = name
        self.price = price
        self.stock = stock

    def final_price(self):
        return self.price

    def sell(self, quantity):
        if quantity > self.stock:
            print(f"Only {self.stock} of {self.name} left.")
            return False
        self.stock -= quantity
        return True

    def __str__(self):
        return f"{self.name} - {self.final_price():.2f} {self.currency} ({self.stock} in stock)"

class PerishableProduct(Product):
    def __init__(self, name, price, stock, days_left):
        super().__init__(name, price, stock)
        self.days_left = days_left

    def final_price(self):
        if self.days_left <= 2:
            return round(self.price * 0.8, 2) # 20% off when close to expiry
        return self.price

    def __str__(self):
        return f"{super().__str__()} [{self.days_left} day(s) left]"

class ElectronicsProduct(Product):
    def __init__(self, name, price, stock, warranty_months):
        super().__init__(name, price, stock)
        self.warranty_months = warranty_months

    def __str__(self):
        return f"{super().__str__()} [{self.warranty_months} months warranty]"
```

And a quick check that overriding works (run it with `uv run practice/inherit_demo.py` after creating the file):

practice/inherit_demo.py

```
from online_store.models import Product, PerishableProduct

milk = PerishableProduct("Fresh Milk", 3.00, 30, days_left=1)
yogurt = PerishableProduct("Yogurt", 1.80, 12, days_left=5)
notebook = Product("Notebook", 4.50, 50)

print(milk.final_price())
print(yogurt.final_price())
print(notebook.final_price())
print(isinstance(milk, Product))
print(isinstance(notebook, PerishableProduct))
```

OUTPUT

```
2.4
1.8
4.5
True
False
```

WATCH OUT

If you forget `super().__init__(...)` in a subclass, the parent's attributes are never created, and you get an `AttributeError` later. Always call it first.

YOUR TURN

Add a `ClothingProduct` subclass with an extra `size` attribute and a `__str__` that shows it, for example T-Shirt - 15.00 USD (8 in stock) [size M].

CHALLENGE

Override `sell()` in `PerishableProduct` so that an expired product (`days_left <= 0`) cannot be sold: print a message and return `False`. For the normal case, call `super().sell(quantity)` instead of copying the parent's code.

CHECKPOINT

1. What does `super().__init__(name, price, stock)` do?
2. `PerishableProduct` does not define `sell()`. Which `sell()` runs when you call it?
3. Why did we only have to change one line in `CartItem` to support new product types?

Part 8 | Composition or inheritance?

CONCEPT

Beginners often reach for inheritance too early. A safe rule: **use "is a" for inheritance, "has a" for composition, and prefer composition when unsure**. Keep hierarchies shallow (one or two levels).

Pair	Relationship	Design
PerishableProduct and Product	A perishable product is a product	Inheritance
Customer and Cart	A customer has a cart	Composition
Cart and CartItem	A cart has items	Composition
Cart and list	A cart is not a list; it has a list of items	Composition

Looking ahead: an agent **has** a list of tools and **has** instructions. It is not a kind of tool. Most agent systems are built from composition.

YOUR TURN

For each pair decide: inheritance or composition? Write one sentence explaining why.

1. Car and Engine
2. Dog and Animal
3. Library and Book
4. Teacher and Person
5. Order and Customer

CHECKPOINT

1. Why would `class Cart(list)` be a risky design for our store?

Part 9 | Reusable class design

CONCEPT

1. **One class, one job.** `Cart` manages cart contents. It does not know how customers pay.
2. **Validate in the constructor.** Refuse bad data at the door by raising an error (see below).
3. **Small methods with verb names:** `sell`, `restock`, `total`. Easy to read, easy to test.
4. **Ask, do not reach in.** Call `cart.total()` instead of looping over `cart.items` from outside.
5. **Internal details start with an underscore** (`_helper`). It is a convention that says "not for outside use".
6. **One module per group of related classes** (`models.py`, `cart.py`, `orders.py`).

Encapsulation and abstraction in practice

Two of the four ideas live here. **Encapsulation:** an object keeps its data and the methods that use it together, and outside code goes through those methods. **Abstraction:** the user of a class needs to know what `cart.total()` does, not how it adds things up. Python has no hard locks, only naming conventions that tell other programmers what is internal:

Style	Example	Meaning
public	<code>name</code>	Free to use from anywhere
protected (convention)	<code>_rooms</code>	One underscore: for internal use. Python does not stop you, but you should not touch it
private (name-mangled)	<code>__safe_code</code>	Two underscores: Python renames it inside the class, so outside code cannot reach it by accident

practice/access_demo.py

```
class House:
    def __init__(self, owner, safe_code):
        self.owner = owner          # public: anyone may read it
        self._rooms = ["kitchen"]   # protected (convention): for internal use
        self.__safe_code = safe_code # private: Python renames it internally

    def open_safe(self, code):
        if code == self.__safe_code:
            return "Safe opened"
        return "Wrong code"

house = House("Sara", 4821)
print(house.owner)
print(house.open_safe(1111))
print(house.open_safe(4821))

try:
    print(house.__safe_code)
except AttributeError as error:
    print("Error:", error)
```

OUTPUT

```
Sara
Wrong code
Safe opened
Error: 'House' object has no attribute '__safe_code'
```

TIP

For this course, the habit that matters most is the single underscore for internal helpers and attributes. Use `__private` rarely.

Raising an error stops the program at the point of the mistake, with a clear message. `try / except` lets the caller handle it instead of crashing:

practice/validation_demo.py

```
from online_store.models import Product

try:
    bad = Product("Notebook", -5, 10)
except ValueError as error:
    print("Could not create product:", error)
```

OUTPUT

```
Could not create product: Price cannot be negative
```

YOUR TURN

Make `Cart.add()` raise `ValueError("Quantity must be at least 1")` when the quantity is below 1. Test it with a `try / except` like the one above.

CHECKPOINT

1. What is the difference between `_rooms` and `__safe_code`?
2. Which two of the four OOP ideas does `cart.total()` illustrate, and why?

Part 10 | Final project: online store checkout

PROJECT

Turn everything into one clean, multi-file application inside your uv package. Your goal is to reproduce the expected output below with your own code.

Target structure

```
online-store/
|-- pyproject.toml
|-- practice/           all your practice files
`-- src/online_store/
    |-- __init__.py
    |-- models.py        Product, PerishableProduct, ElectronicsProduct (Part 7)
    |-- cart.py          CartItem, Cart (your shopping cart, updated)
    |-- orders.py        Customer, Order (Part 6)
    `-- main.py          the demo below
```

Step by step

1. Create `models.py` from Part 7 (include the validation from Part 9 and a `currency = "USD"` class attribute on `Product`).
2. Create `cart.py` from your shopping cart project (Part 5). Two changes: `CartItem.subtotal()` must call `self.product.final_price()`, and `Cart.checkout()` must return `(purchased, total)` (Part 6).
3. Create `orders.py` from the starter below.
4. Create `main.py` exactly as shown.
5. Register the script. In `pyproject.toml`, change the `[project.scripts]` line to point at your new `main()` function, then run it.

pyproject.toml

```
[project.scripts]
online-store = "online_store.main:main"
```

NOTE

Inside the package, modules import each other with the package name: `from online_store.cart import Cart`. The folder is `online_store` (underscore), the project name is `online-store` (hyphen).

src/online_store/orders.py (starter)

```

from online_store.cart import Cart

class Order:
    order_count = 0          # class attribute: counts every order created

    def __init__(self, customer, items, total):
        # TODO 1: increase Order.order_count by 1, then store order_id
        #           (the new count), customer, items and total on the object
        ...

    def print_receipt(self):
        # TODO 2: print the receipt (match the expected output)
        ...

class Customer:
    def __init__(self, name, email):
        # TODO 3: store name and email; give the customer their own Cart
        ...

    def place_order(self):
        # TODO 4: check out the cart (it returns items and total)
        #           and return a new Order for this customer
        ...

```

src/online_store/main.py

```

from online_store.models import Product, PerishableProduct, ElectronicsProduct
from online_store.orders import Customer

def main():
    notebook = Product("Notebook", 4.50, 50)
    milk = PerishableProduct("Fresh Milk", 3.00, 30, days_left=1)
    headphones = ElectronicsProduct("Headphones", 59.99, 10, warranty_months=12)

    for product in (notebook, milk, headphones):
        print(product)
    print()

    sara = Customer("Sara", "sara@example.com")
    sara.cart.add(notebook, 4)
    sara.cart.add(milk, 3)
    sara.cart.add(headphones, 1)
    sara.cart.show()
    print()

    order = sara.place_order()
    order.print_receipt()
    print()

    print(notebook)
    print(milk)
    print(headphones)

```

Expected output

```
uv run online-store
```

```
Notebook - 4.50 USD (50 in stock)
Fresh Milk - 2.40 USD (30 in stock) [1 day(s) left]
Headphones - 59.99 USD (10 in stock) [12 months warranty]
```

```
--- Cart ---
4 x Notebook      18.00
3 x Fresh Milk    7.20
1 x Headphones    59.99
Total             85.19
```

```
=== Receipt #1 ===
Customer: Sara <sara@example.com>
4 x Notebook      18.00
3 x Fresh Milk    7.20
1 x Headphones    59.99
Total             85.19
```

```
Notebook - 4.50 USD (46 in stock)
Fresh Milk - 2.40 USD (27 in stock) [1 day(s) left]
Headphones - 59.99 USD (9 in stock) [12 months warranty]
```

What your project must demonstrate

Requirement	Where
Inheritance with <code>super()</code> and an overridden method	<code>models.py</code>
Composition: Cart has CartItems; Customer has a Cart	<code>cart.py</code> , <code>orders.py</code>
A class attribute used as a counter	<code>Order.order_count</code>
Validation that raises <code>ValueError</code>	<code>Product.__init__</code> , <code>Cart.checkout</code>
The <code>main()</code> pattern and a registered script	<code>main.py</code> , <code>pyproject.toml</code>

CHALLENGE

Pick one extension

- **Coupons:** a `Coupon(code, percent)` class and `Cart.apply_coupon(coupon)` that reduces the total.
- **Inventory:** an `Inventory` class that holds all products, with `find(name)` and a `low_stock_report()` that lists products at or below their stock limit.

Part 11 | Where this shows up next

CONCEPT

Everything you wrote this week reappears in the next weeks, often with fewer lines because the libraries write the boilerplate for you.

This week	Coming up
<code>class Product</code> with attributes and <code>__init__</code> validation	FastAPI and SQLAlchemy models: fields with types, validated automatically (Week 5)
An object that holds other objects (Customer has a Cart)	An agent that holds instructions and a list of tools (Weeks 7 and 8)
Overriding a method in a subclass	Custom behaviour in tools, guardrails and workflows (Weeks 8 to 11)
Registered script in <code>pyproject.toml</code>	Running your own agent apps with <code>uv run</code>

Here is a toy version of the agent idea, written with only what you know now. It is **not** the real SDK, but real agent frameworks have the same shape: an `Agent` object with a name, instructions and a list of tool objects.

```
practice/agent_preview.py

class Tool:
    def __init__(self, name, description, func):
        self.name = name
        self.description = description
        self.func = func

    def run(self, **kwargs):
        return self.func(**kwargs)

class Agent:
    def __init__(self, name, instructions, tools=None):
        self.name = name
        self.instructions = instructions
        self.tools = tools or []

    def find_tool(self, name):
        for tool in self.tools:
            if tool.name == name:
                return tool
        return None

    def get_price(product_name):
        prices = {"notebook": 4.50, "backpack": 35.00}
        return prices.get(product_name.lower(), "unknown product")

price_tool = Tool("get_price", "Look up the price of a product", get_price)
shop_agent = Agent(
    name="Shop Assistant",
    instructions="Help customers find product prices.",
    tools=[price_tool],
)

tool = shop_agent.find_tool("get_price")
print(tool.run(product_name="Notebook"))
print(shop_agent.find_tool("refund"))
```

```
OUTPUT

4.5
None
```

And a peek at a model class in the next week's style (preview only, do not run it yet):

```
class Product(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    price: float
    stock: int
```

Reference card: composition and inheritance

You want to...	Write
Make a subclass	<code>class PerishableProduct(Product):</code>
Call the parent constructor	<code>super().__init__(name, price, stock)</code>
Extend a parent method	<code>return super().__str__() + " [extra]"</code>
Override a method	define a method with the same name in the subclass
Check the type of an object	<code>isinstance(milk, Product) # True for subclasses too</code>
Mark something internal	<code>self._rooms = []</code> (one underscore)
Name-mangled private	<code>self.__safe_code = code</code> (two underscores)
Raise an error	<code>raise ValueError("message")</code>
Handle an error	<code>try: ... except ValueError as error: ...</code>
Import from your package	<code>from online_store.cart import Cart</code>

Checkpoint answers

Try each checkpoint first. Then compare. Your wording can differ; the idea should match.

Part 1

1. Keys can be mistyped and fail only at runtime; nothing guarantees every dictionary has the same fields; the data and the functions that use it are not tied together.
2. A class is the blueprint (it describes what every house will have). An object is one real house built from it. One blueprint can produce many different houses.
3. Each object stores its own attribute values. Painting `house_2` changes only `house_2`, even though both houses came from the same blueprint.
4. Encapsulation (the front door, not the walls), abstraction (the switch without the wiring), inheritance (the villa blueprint built on the house blueprint), polymorphism (one action, different behaviour per house). Any one matched to its house example is fine.

Part 2

1. The specific object the method is working on (for `notebook.sell(10)`, it is `notebook`).
2. Automatically, every time you create an object with `Product(...)`.
3. `TypeError: ... takes 0 positional arguments but 1 was given`. Python always passes the object as the first argument, so the method must have `self`.

Part 3

1. `PKR`. Only `notebook` got its own `currency`; `backpack` still reads the class attribute.
2. The list is shared by every object of the class, so one object's changes appear in all of them.

Part 4

1. `CartItem`, because it is the only object that knows both the product and the quantity.
2. `cart.items` holds two `CartItem` objects. The third add was refused, and the quantity is stored inside an item, not counted as a separate line.

Part 6

1. Customer has a Cart. Order has a Customer. Cart has CartItems.
2. Each customer needs a separate cart. A shared cart would mix up everyone's items.

Part 7

1. It runs the parent class's constructor, so the parent's attributes are set up without repeating its code.
2. The one inherited from `Product`.
3. Because `final_price()` exists on every product type, `CartItem` can call it without knowing the type (this is called polymorphism).

Part 8 Your Turn

1. Car and Engine: composition (a car has an engine).
2. Dog and Animal: inheritance (a dog is an animal).
3. Library and Book: composition (a library has books).
4. Teacher and Person: inheritance (a teacher is a person).
5. Order and Customer: composition (an order has a customer).

Part 8 Checkpoint

1. `Cart(list)` would inherit dozens of list methods (`insert`, `sort`, `pop`...) that bypass our stock checks, and a cart is not really a list. Composition keeps control of what can be done.

Part 9

1. `_rooms` (one underscore) is a convention meaning internal use; Python does not block access.
`__safe_code` (two underscores) is renamed inside the class, so outside code cannot reach it by accident.
2. Abstraction, because the caller only needs to know what `total()` does, not how. Encapsulation, because the cart's data (`items`) is used through its own methods rather than from outside.